*Document number: P1321R0*
*Audience: EWG, SG12*

*Ville Voutilainen*
*2018-10-07*

---

# UB in contract violations

## Abstract

[dcl.attr.contract.check]/4:

> During constant expression evaluation (7.7), only predicates of checked contracts are evaluated. In other contexts, it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to false, the behavior is undefined.

This seems like an open hunting license for optimization: A compiler can probably assume that a predicate evaluates to true and optimize based on that. Perhaps even worse, a compiler can sometimes see that a predicate evaluates to false (but is not evaluated in the current checking mode), and do anything.

## Really?

Well, this is courtesy of Peter Dimov:

https://godbolt.org/g/7TP7Mt

Peter further explains:

```
test():
  sub rsp, 8
  call rand
  mov esi, 2
  mov edi, OFFSET FLAT:.LC0
  xor eax, eax
  call printf
.L22:
  call bar()
  add rsp, 8
  ret
```

Since the not-checked [[expects audit: x==2]] is assumed to pass, x is replaced with the constant 2 and all other checks are optimized out.

Peter continues:

If you make `f`

```
void f(int x) [[expects audit: x>=1 && x<=2]]
```

and add `extern` to `a` in `g`

```
void g(int x) [[expects: x>=0 && x<3]]
    {
        extern int a[3];
        a[x] = 42;
    }
```

your (Herb's) point becomes clearer, as now `test` does write 42 to a random location:

https://godbolt.org/g/SfKWPu-2

The background of that exploration is this write-up by Herb:

Since we now agree about my multi-level contract example this morning about UB when only some contracts are enabled… carefully coming back to my earlier example, which I agree did not allow magic before, but now changing the first precondition to "audit" or "axiom":

```
// same as before, except now "XXX" contract level on f, see bullets below

void f(int x) [[expects XXX: x==2]] { ... }

void g(int x) [[expects: x>=0 && x<3]] { int a[3]; a[x] = 42; }

void h(int x) [[expects: x>=1 && x<=3]] { switch(x) { case 1: foo(); case 2: bar(); case 3: baz(); } }
```

And we run them in series approximately like this:

```
int val = std::rand();

try { f(val); /*...*/ } catch(...) { /*...*/ }

try { g(val); /*...*/ } catch(...) { /*...*/ }

try { h(val); /*...*/ } catch(...) { /*...*/ }
```

then if either of these two cases is true:

- if "XXX" is "audit", and the test harness enables "default" checks; or
- if "XXX" is "axiom", and the test harness enables both "default" and "audit" checks (== "maximum checks" because I keep being assured that axioms are intended never to be checked);

then do we agree this allows all the magic and UB I enumerated before (wild write + wild branch), including that as before the two "enabled" checks on g and h can be elided and not fire?

# Is this a problem?

First of all, I'm not sure. Second, I don't know what we intend with the UB in the specification of a contract check that is not evaluated in the current checking mode but might be evaluated anyway. What concerns me is that if Peter's contracts emulation does what an actual contract implementation would do, I need to be very VERY careful about never EVER writing a contract that would evaluate to false unless I'm sure that it would evaluate to false only in the current checking mode, and I also need to be very careful about writing contracts that would evaluate to true in any other mode than the current checking mode, because I wouldn't want the non-current-mode contracts end up optimizing away my contracts for the current-mode contracts I have. I'm not sure whether those are the only things I need to be careful about.

I'm not sure I know how to be so careful, especially when I'm not sure what I need to be careful about.

In an email discussion about this, here's what I wrote:

"It boils down to this: an unchecked predicate that evaluates to false is UB, if the compiler decides to lay down code that evaluates that predicate in the mode that wouldn't otherwise evaluate it. So combinations of different checking levels on the same function or even the same translation unit (or in the same program, even) can trigger magical powers. Using just one level of checking can do that too without any problems; put just audit-level contracts into your program, run in default mode. You can go boom, boom, *BOOM*."

# If it is a problem, what should we do?

Well, I would entertain some ideas:

1. I want to be able to expect (no pun intended) that when I write a contract predicate, its evaluation has no effect on the evaluation of other contracts predicates, whether any of those predicates are evaluated or not.
2. I want to be able to expect that when I write a contract predicate that is evaluated in some standard checking mode, its evaluation doesn't cause UB in any standard checking mode just because it ends up evaluating to false, and that its evaluation doesn't affect the evaluation of other contract predicates.